# 代码染色&无效代码清理

原创 首页&信息流工程 大淘宝技术 2025年11月21日 01:44 浙江 314人



D是我们团队的服务端应用，其代码库历史悠久，最早可以追溯到淘宝APP无线端迁移，应用中许多代码已无线上流量，但代码并未随业务的下线被清理。越来越多的代码“沉淀”下来，既增加了团队新人学习门槛，也增加日常开发维护成本。但实际做代码下线并非容易，仅凭业务逻辑决策代码清理费时费力，还容易误删在使用的业务代码，因此非常需要工具来辅助做代码的清理，这就是基于代码执行染色和覆盖分析做代码下线方案的背景。

代码执行染色&执行覆盖率分析，使用JVM agent的扩展能力实现代码的插桩和在线染色，再通过解析采样的数据可得到代码的执行情况，清理代码就“有理有据”；仅靠原始分析出的数据清理依然低效，为此我们将数据采集、覆盖率可视化通过IDEA插件集成，实现清理无效代码过程又准又快。

## 01 代码覆盖率采集

### 1.1 JVM Agent 概述

在 Java 中，插桩(Instrumentation) 是一个非常重要的接口，它为开发者提供了在 JVM 运行时动态修改类字节码的能力。这个功能是 Java 提供的 Java Agent 机制的核心组成部分，常用于性能监控、AOP（面向切面编程）、代码覆盖率分析、热部署等场景。
Instrumentation 接口位于java.lang.instrument包中，由 JVM 在加载 Java Agent 时自动提供一个实例，通过预定义的方法传递给 Agent。
主要用途包括：

- 动态修改类的字节码（Class File）
- 获取 JVM 中已加载的所有类
- 添加类文件转换器（ClassFileTransformer）
- 重新定义类（redefineClasses）
- 重置类的字节码（reset class definitions）

对在线代码的插桩须通过 Java Agent 机制在 JVM 启动时或运行时介入类加载过程，它可以对 JVM 进行一些操作，如修改类字节码、监控 JVM 状态等。Agent 可以通过 –javaagent 参数在启动时加载，也可以通过 Attach API 在运行时动态加载。

| 类别 | 依赖重启 | 长期采集 | 资源占用 |
| --- | --- | --- | --- |
| agent | 是 | 稳定 | 共用jvm |
| attach | 否 | 重启失效 | 独立jvm |

- 使用agent方式是在jvm启动参数指定 –javaagent 来加载agent jar包，agent需要实现AgentMain接口。
- 使用attach的方式会创建独立JVM，占用独立内存资源，使用attach的方式将插桩jar包作用于目标jvm，插桩jar包需要实现PreMain的接口，代码示例：

```
String jarFile = args[0];
String pid = getPid(args);
logger.info("Attaching agent to PID: " + pid);
VirtualMachine vm = null;
try {
  vm = VirtualMachine.attach(pid);
  vm.loadAgent(jarFile);
  logger.info("Agent attached successfully");
} catch (IOException ioException) {
  logger.critical("load agent jar fail: " + jarFile);
} catch (AttachNotSupportedException attachNotSupportedException) {
  logger.critical("attach to jvm fail: " + pid);
} catch (AgentLoadException | AgentInitializationException agentExcept
  logger.critical("jvm load agent or agent init fail: " + pid);
} finally {
  if (vm!=null) {
    vm.detach();
  }
}
```

两种方式因使用方式不同，适用场景会有差异，后续会做对比介绍。

### 1.2 代码执行覆盖

统计代码执行情况可以基于覆盖率，有行覆盖率、分支覆盖、方法覆盖等概念。针对腐朽代码染色我们主要讨论行覆盖率，知道指定package和类的行覆盖率就大概可以知道治理的方向。

- 1.2.1 自定义方案

自定义插桩主要基于1.1中agent或者attach的方式，通过1.2中的接口和ASM实现代码的插桩，实际插桩可以按行执行，也可以按方法插桩，示例可以参考1.2.3中。自定义的方式对理解和实践插桩过程很有帮助，比较方便自定义的方式处理采集的数据，但存在问题：

- 执行效率低，尤其是按行插桩，侵入性太大；
- 并发情况下存在锁竞争风险，虽然实际结果对统计代码执行覆盖没影响；
- 执行结果使用上不便于集成IDEA，对代码覆盖的可视化支持难度大；

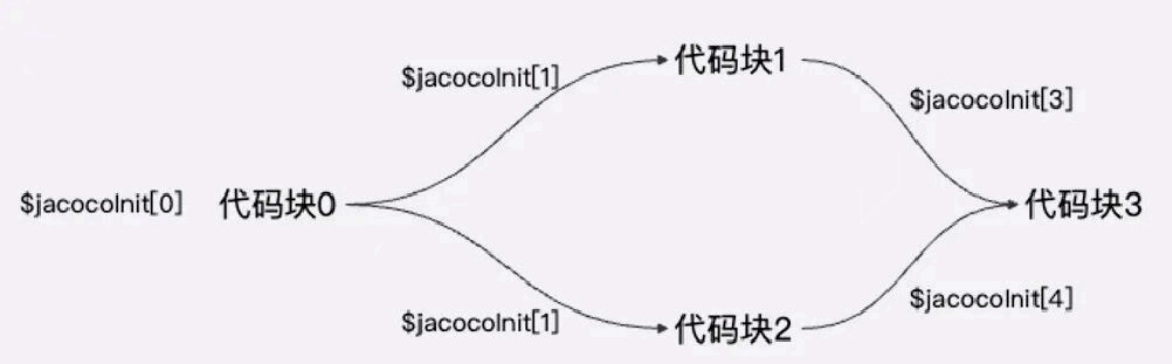
- 1.2.2 JaCoCo方案

JaCoCo (Java Code Coverage)通过在类加载时修改字节码来插入探针（probes），这些探针会在代码执行时记录哪些代码行被访问过，当类被加载时，JaCoCo 的 ClassFileTransformer 会被调用，会修改类的字节码，插入探针来记录代码的执行情况。

JaCoCo 使用的探针为 $jacocoInit[boolean] 数组，数组长度为类的探针数量，由CFG(Control Flow Graph， 控制流图)分析得到，将代码划分为代码块，$jacocoInit的每个位置代码一个代码块入口，$jacocoInit[n]=true则表示整个代码块都被执行。

```
public void exampleMethod(int a, int b) {
  // do some thing 代码块0
  if(a > 0){
    // do some thing 代码块1
  }else if(b > 0){
    // do some thing 代码块2
  }
  // do some thing 代码块3
}
```

上图代码示例中，有多个代码块，在CFG中为一个节点，CFG的边为代码块执行流转关系，上图代码插桩后的探针如下图:




通过按代码块的插桩模式，比逐行记录代码执行情况高效很多，并且使用布尔数组的方式判断代码是否执行也非常高效(如果使用数值变量累加会存在锁的问题，对代码执行性能影响较大)。JaCoCo提供了agent和cli工具，使用方式可参考：JaCoCo agent和JaCoCo cli。

## 1.3 采集方案对比

### 1.3.1 自研插桩 vs JaCoCo工具

高效将探针插入到代码中，不对正常业务代码造成太大影响，是保障线上代码正常采集结果的前提，如前所述，自己实现插桩自定义空间大，方案设计灵活，缺点是效率和开发成本高；使用JaCoCo工具可以快速集成、性能高效。

| | 优点 | 缺点 |
| --- | --- | --- |
| 自研插桩 | 灵活性高、数据处理方便 | 开发成本、稳定性验证 |
| JaCoCo工具 | 稳定高效、快速集成 | 数据格式固定，采集数据需二次加工 |

考虑到目标应用D对稳定性和采集性能要求高，使用JaCoCo的方案更加满足我们的需求。

### 1.3.2 agent vs attach

在1.1中有介绍两者的区别，各有优势，对比如下:

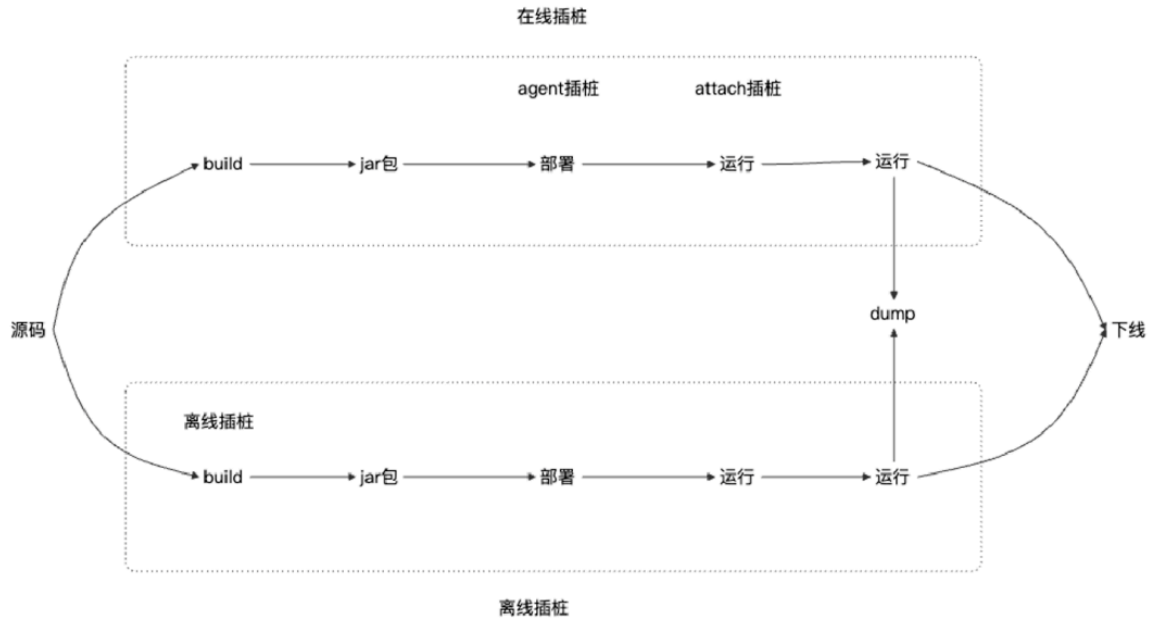
| | 使用方式 | 性能影响 | 重启影响 | 卸载插桩 |
| --- | --- | --- | --- | --- |
| agent方式 | jvm启动参数增加agent参数 | 随jvm一起启动，业务无感，平均cpu会上涨 | 重启不失效 | 重新发布 |

| | | | | |
|---|---|---|---|---|
| attach方式 | 独立jvm attach到业务jvm | attach的时候触发代码插桩会导致jvm飙升，之后下降后稳定 | 重启失效 | 无需重启 |

通过对比可知，agent的方式对代码有一定入侵，适合长期稳定采集代码覆盖数据；attach的方式则更加灵活，适合临时做单机的代码采样分析，参考arthas做在线分析的方式。

- 1.3.3 在线vs离线

针对包含热部署的场景，还会涉及到在线和离线插桩的选择。在jvm启动后做代码字节增强的算是在线插桩，而离线插桩指在代码编译打包且尚未部署阶段，通过maven插件的方式对代码做修改。具体而言，就是在mvn build的时候引入JaCoCo的插件，直接在build阶段将字节码插桩，这样部署的时候就已经是插桩的代码。




如上图，在线插桩在部署和运行阶段，离线插桩在构建阶段。以使用JaCoCo做离线插桩为例，还需要在部署时引入JaCoCo的依赖，否则会出现java.lang.TypeNotPresentException，表示在类初始化时找不到依赖类，需要引入下面这个依赖：

```
<!-- https://mvnrepository.com/artifact/org.jacoco/org.jacoco.agent -->
<dependency>
    <groupId>org.jacoco</groupId>
    <artifactId>org.jacoco.agent</artifactId>
    <version>0.8.12</version>
```

```
    <scope>runtime</scope>
    <classfier>runtime</classfier>
</dependency>
```

注意必须用<classfier>标签，生效的包是org.jacoco.agent-0.8.12-runtime.jar，而不是org.jacoco.agent-0.8.12.jar，包含runtime的包包含完整的依赖。

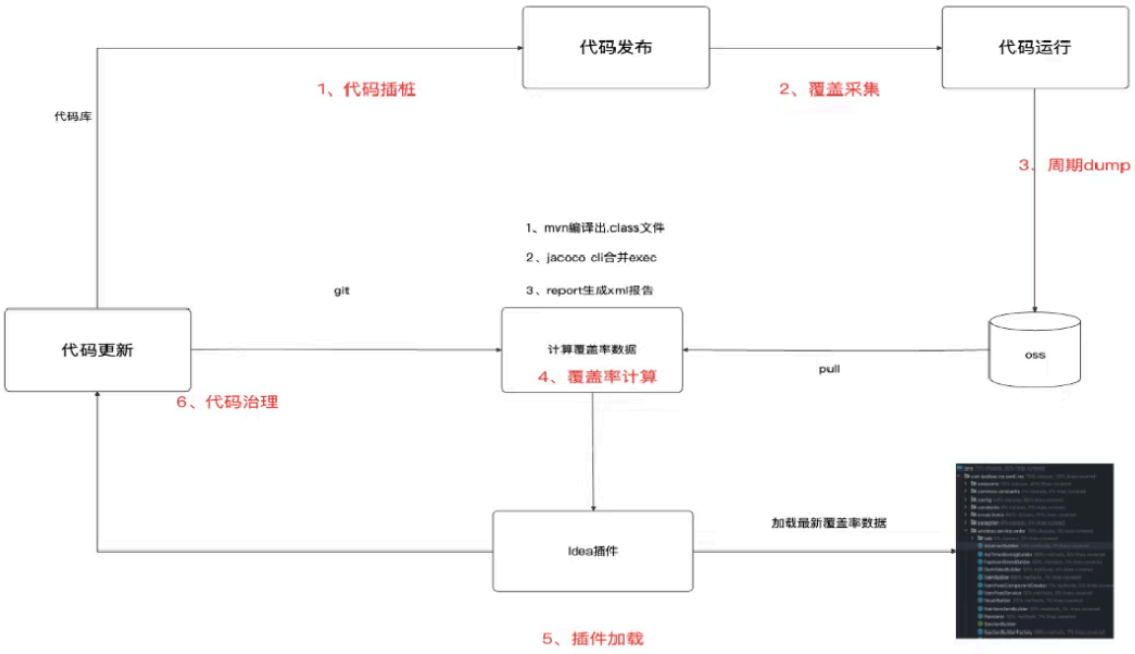
这种方式适合热部署的插桩，对插桩的包选择性部署。离线方案的缺点在于，考虑到插桩代码运行存在性能损耗，如果不是所有在线机器都部署采集，每次部署就需要至少打出两份部署包：插桩版本的包和不做插桩的包，将插桩包部署到需要采集的机器，非插桩包部署到其它机器，这种分别部署的方式增加了部署调度的复杂性。

### 1.4 方案选择

D应用的代码治理是长期持续的过程，我们需要长期周期性持续采集在线代码执行数据，同时要考虑稳定性和效率，最终选择以agent的方式落地。我们复用了JaCoCo的采集染色方案，这一方案成熟可靠，对系统运行时影响较低。通过在docker file集成JaCoCo依赖，在环境脚本中为jvm指定agent参数，并选择性采集安全生产环境，实现了对在线代码执行数据的采样采集。




核心工作链路如下：




整个采集+治理流程：

1. 代码插桩：在线或者离线方式对业务代码完成探针插入；
2. 覆盖采集：业务代码跑起来，让每个QPS的请求渗透到每行代码，完成代码染色；
3. 周期dump：代码经过充分运行后，周期将最新的代码覆盖数据dump到对象存储OSS中；
4. 覆盖率计算：覆盖率包含行、分支、类覆盖等，依赖代码.class文件和dump的采集数据，输出xml覆盖报告；
5. 插件加载：每次打开IDEA项目代码时，加载最新的覆盖报告数据；
6. 代码治理：形成治理闭环，根据最新的报告清理或者重构代码，提升代码健康度。

## 2.1 整体设计

代码覆盖率主要针对日常开发代码链路检查、为代码重构、代码清理提供可靠依据，方便代码执行情况的检查。

从功能上应具备以下功能：

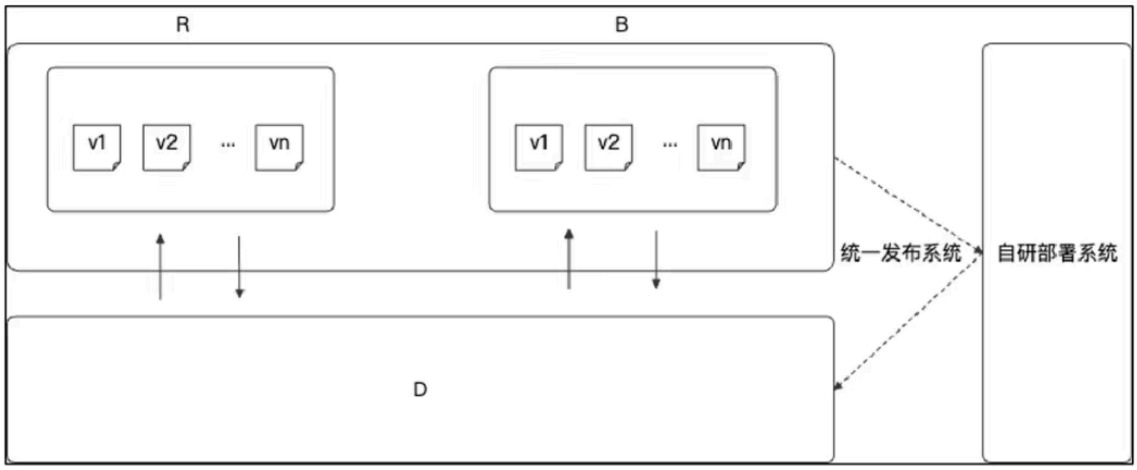
- **代码采集**：支持应用代码执行覆盖率采集(主要为.exec文件)，不影响原有业务逻辑；
  - 基于agent的方式，持续周期性采集；
  - 支持长期数据文档采集，合并多天数据；
- **数据合并：**代码执行情况的采集数据与最新代码版本生成完整代码覆盖情况；(最终代码覆盖=.exec文件+.class文件)
- **IDEA插件：**代码执行情况可视化，IDEA插件支持目标应用代码执行情况的可视化
  - IDEA目标项目检测，非目标项目提醒；
  - 目标项目打开自动下载oss覆盖数据，也可配置第一次使用时下载；
  - 支持多天数据采集下载和采集结果合并，提高代码采集覆盖准确性；

- 打开代码执行覆盖情况，展示package、class执行覆盖率和代码行级执行情况；
- 隐藏代码执行覆盖情况，关闭package、class执行覆盖率和代码行级执行情况的展示；
- 支持采集数据的缓存和刷新，无需重复下载oss数据；
- 右键和工具栏支均支持打开&关闭展示、工具栏和配置页支持插件相关的配置；

## 2.2 代码采集

### 2.2.1 热部署的影响

在应用D之上基于自研热部署方案部署了R和B应用，其关系是这样的：




D应用就像个容器，里面主要定义了基础的流程；业务通常以插件的方式在R和B中做单独开发迭代，这样就可以将业务逻辑抽到上层，方便业务的快速发布和迭代。这样的架构对代码插桩染色是否有影响呢？看下D中实例化容器加载R的classLoader的实现：

```
private void initClassLoader(ClassLoader parentClassLoader, File appPa
    try {
        File[] extFile = appPath.listFiles((dir, name1) -> "ext".equal
        AppExtClassLoader extClassLoader = null;
        if (extFile != null && extFile.length > 0) {
            File[] extJars = extFile[0].listFiles((dir, name1) -> name
            extClassLoader = new AppExtClassLoader(parentClassLoader, 
        }
        File unzipFile = Paths.get(path, appName).toFile();
        ZipFileUtils.unzip(new ZipFile(appJar), unzipFile);
        this.classLoader = new AppClassLoader(unzipFile, extClassLoade
    } catch (Throwable e) {
        throw new ContainerException("container constructor error init
```

```
    }
}
```

R和B是按照jar包部署的，初始化classLoader时分别按ext目录和解压后应用目录创建YardAppClassLoader，其父classloader就是D使用的pandora加载业务代码的LaunchedURLClassLoader，也就是说通过自定义类加载的路径，使其在加载热部署代码时按照jar包解压的路径加载代码；不同的部署版本解压出来路径不一样，做到了代码隔离。对于使用agent插桩，不管是D代码还是R、B的热部署代码，生效逻辑并没有差别。实际插桩效果也验证了这样的结论：

D插桩代码反编译：

```
@Service
public class L      Monitor implements Runnable, InitializingBean {
    // $FF: synthetic field
    private static transient boolean[] $jacocoData;

    private static double calcCpuUsage(long[] ticks, long[] prevTicks) {
        boolean[] var3 = $jacocoInit();
        long nice = ticks[TickType.NICE.getIndex()] - prevTicks[TickType.NICE.getIndex()];// 44
        var3[13] = true;
        long irq = ticks[TickType.IRQ.getIndex()] - prevTicks[TickType.IRQ.getIndex()];// 45
        var3[14] = true;
        long softIrq = ticks[TickType.SOFTIRQ.getIndex()] - prevTicks[TickType.SOFTIRQ.getIndex()];// 46
        var3[15] = true;
        long steal = ticks[TickType.STEAL.getIndex()] - prevTicks[TickType.STEAL.getIndex()];// 47
        var3[16] = true;
        long cSys = ticks[TickType.SYSTEM.getIndex()] - prevTicks[TickType.SYSTEM.getIndex()];// 48
        var3[17] = true;
        long user = ticks[TickType.USER.getIndex()] - prevTicks[TickType.USER.getIndex()];// 49
        var3[18] = true;
        long ioWait = ticks[TickType.IOWAIT.getIndex()] - prevTicks[TickType.IOWAIT.getIndex()];// 50
        var3[19] = true;
        long idle = ticks[TickType.IDLE.getIndex()] - prevTicks[TickType.IDLE.getIndex()];// 51
        long idleTicks = ioWait + idle;// 53
        long totalTicks = idleTicks + user + nice + cSys + irq + softIrq + steal;// 54
        var3[20] = true;
        double var10000 = 100.0D - 100.0D * (double)idleTicks / (double)Math.max(totalTicks, 1L);// 56
        var3[21] = true;
        return var10000;
    }
```

R插桩代码反编译：

```
package c                                                    .handler;

import java.lang..JaCoCo;

public class LocalTabInfo {
    boolean isLocalTab;
    String localTabContainerId;
    // $FF: synthetic field
    private static transient boolean[] $jacocoData;

    public void setLocalTab(boolean localTab) {
        boolean[] var3 = $jacocoInit();
        this.isLocalTab = localTab;// 19
        var3[2] = true;// 20
    }

    public void setLocalTabContainerId(String localTabContainerId) {
        boolean[] var3 = $jacocoInit();
        this.localTabContainerId = localTabContainerId;// 27
        var3[4] = true;// 28
    }

    public boolean isLocalTab() {
        boolean[] var2 = $jacocoInit();
        var2[1] = true;
        return this.isLocalTab;// 15
    }

    // $FF: synthetic method
    private static boolean[] $jacocoInit() {
        boolean[] var10000 = $jacocoData;
        if ($jacocoData == null) {
            Object var0 = JaCoCo.data;
            Object[] var10002 = new Object[]{-8746073972038152057L, "                                                    /handler/LocalTabInfo", Integer.valueOf(5)};
            var0.equals(var10002);
            var10000 = $jacocoData = (boolean[])var10002[0];
        }
```

- 2.2.2 代码改造

D采用agent方式，在应用启动时同步启动jvm agent并常驻内存。主要改动docker file文件：

step1: 在基础镜像中下载JaCoCo agent的文件，并放到/home/admin/app/jacoco-runtime.jar下：

```
wget -c -O /home/admin/app/jacoco-runtime.jar "https://repo1.maven.org/r
```

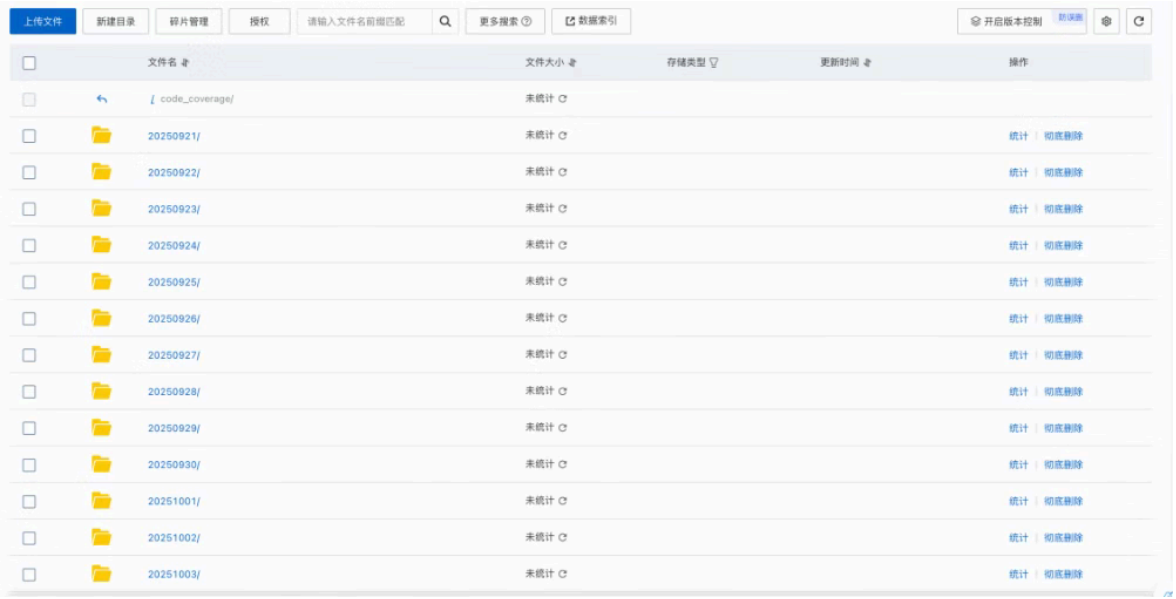
step2: 在环境配置脚本中配置jvm启动参数：将JaCoCo agent参数放到-javaagent中，主要核心参数包括agent的路径和插桩范围的白名单。这里将应用D、R和B代码相关的类路径都放进去，非白名单文件不会被插桩，这能控制插桩范围，降低插桩对应用运行时的影响。这里我们限制了预发和安全生产环境，通过采集分组隔离避免影响正式分组。

```
JACOCO_AGENT="/home/admin/app/jacoco-runtime.jar"
# 使用白名单，如何使用黑名单且黑名单失效会导致所有场景都生效
if [[ $                                    == "PRE_PUBLISH" || $                                    == "SMALLFLOW" ]]; then
    if [ -e $JACOCO_AGENT ]; then
        # 过滤指定插桩的包，如果新增路径可能会失效
        CLASS_FILTER="com.taobao.t                    *:com.taobao.                    .*:com.taobao.                    .*:com.taobao.                    .*:com.taobao.                    .*:com.taobao
        AGENT_OPT="-javaagent:${JACOCO_AGENT}=includes=${CLASS_FILTER}"
        SERVICE_OPTS="$SERVICE_OPTS $AGENT_OPT"
    fi
fi
```

step3: 数据dump。在应用部署后JaCoCo已经对代码做插桩处理，类代码执行时会自动记录采集的数据到boolean[] jacocoData数组中，数据会一直保存到下次重启，为了周期性将数据从内存采集下来分析，还需要定时dump数据。

```java
/**
* 使用jacoco的数据结构做dump，返回dump是否成功
*
* @param filePath dump的位置
* @return 是否执行成功
*/
boolean jacocoDump(String filePath) throws IOException {
  Agent iAgent = Agent.getInstance();
  if (iAgent == null) {
    DosaLogUtil.warnNew("Jacoco agent not found!");
    return false;
  }
  AgentOptions agentOptions = buildOptions(filePath);
  FileOutput fileOutput = new FileOutput();
  fileOutput.startup(agentOptions, iAgent.getData());
  fileOutput.writeExecutionData(true);
  return true;
}
```

上述代码复用了JaCoCo的FileOutput类，JaCoCo也是依赖这个类的writeExecutionData()实现采集数据写磁盘。采集的数据dump后上传到oss，按照日期做了归档。dump的周期是每天凌晨，这样保证对dump数据做分析时代码是最新部署的。




至此我们实现了插桩并采集了应用D的安全生产环境的代码执行情况，考虑了自定义部署的情况，下面介绍采集后数据的使用。

## 2.3 数据合并

JaCoCo采集的是类的boolean[] jacooData数据，一般用.exec后缀的文件保存。正如前面介绍，jacooData是按代码块标识的数组，并没有类信息，也就是说直接从JaCoCo采集的数据无法计算类代码覆盖率，必须配合类的.class文件使用，因此就有：

**详细执行覆盖率(.xml) = 类插桩执行文件(.exec) + 类原始编译文件(.class)**

IDEA支持多种覆盖率数据，这里选择xml文件来记录代码详细执行情况。想要获取详细覆盖率数据，我们要：

step1: 从oss下载采集的插桩代码执行文件；

step2: git下载最新master代码，编译生产.class文件；

step3: 使用JaCoCo的report功能，实现项目覆盖率的生成和导出；

step4: 上传覆盖率数据到oss，并清理代码缓存。

上述步骤中oss上传和下载不再赘述，主要介绍代码git、maven编译和覆盖率report的方案。

- 2.3.1 jGit代码到本地

java版本的git客户端jGit能实现代码的克隆，而我们这里使用了库token的方式，其优势是不使用个人账号，不支持代码推送，满足我们对代码.class文件的提取需求。

```
/**
* 克隆代码仓库
*
* @param config 应用配置
* @return 本地仓库路径，失败返回null
*/
public String cloneRepository(CodeProfilerAppConfigDO config) {
  String appName = config.getAppName();
  String localRepoPath = buildLocalRepoPath(appName);
  try {
    String ciToken = kcUtil.decrypt(config.getCiToken());
    GitCloneRequest cloneRequest = new GitCloneRequest()
      .setRepoUrl(config.getGitUrl())
      .setBranch(config.getDefaultBranch())
      .setTargetDir(localRepoPath)
      .setCiToken(ciToken);
    GitHelper.clone(cloneRequest);
    return localRepoPath;
  } catch (Exception exception) {
    LOGGER.error("cloneRepository:clone failed for app: " + appName, e
    return null;
  }
}
```

库token配置是应用维度的，具体配置加密存在配置中间价，使用时再拿出来解码使用，避免泄漏。

- 2.3.2 本地跑maven

对已经下载的代码库，可以使用ProcessBuilder实现maven编译，但前提是服务器安装配置了maven。在base docker file中添加maven和maven settings.xml的下载，这样就可以在服务器上用命令行的方式执行maven命令。

```
wget http://mirrors.aliyun.com/apache/maven/maven-3/3.9.11/binaries/apache-maven-3.9.11-bin.tar.gz -O /tmp/apache-maven-3.9.11-bin.tar.gz && \
tar -xvf /tmp/apache-maven-3.9.11-bin.tar.gz -C /opt && \
curl https://y[redacted].aliyuncs.com/maven-settings/settings-online.xml > /opt/apache-maven-3.9.11/conf/settings.xml &&\
rm /tmp/apache-maven-3.9.11-bin.tar.gz && \
```

实际使用时使用了java.lang.ProcessBuilder，其提供了使用java执行shell命令的能力，核心代码：

```
1  private static final String MAVEN_CMD = "/opt/apache-maven-3.9.11/bin/
2  private static final String MAVEN_COMPILE = "compile";
3  private static final String MAVEN_SKIP_TESTS = "-DskipTests=true";
4  private static final String MAVEN_TEST_SKIP = "-Dmaven.test.skip=true"
5  private static final String MAVEN_AUTO_CONFIG_INTERACTIVE = "-Dautocon
6  private static final String MAVEN_PROJECT_BUILD_SOURCE_ENCODING = "-Dp
7  /**
8   * Maven编译项目
9   *
10  * @param config 应用配置
11  * @param localRepoPath 本地仓库路径
12  * @return 成功返回true，失败返回false
13 */
14 public boolean compileProject(CodeProfilerAppConfigDO config, String l
15   String appName = config.getAppName();
16   String[] commands = new String[] {
17     MAVEN_CMD, MAVEN_COMPILE, MAVEN_SKIP_TESTS, MAVEN_TEST_SKIP, MAVEN_
18     MAVEN_PROJECT_BUILD_SOURCE_ENCODING
19   };
20   try {
21     int exitCode = MavenHelper.execute(commands, localRepoPath);
22     if (exitCode == 0) {
23       LOGGER.info("maven build succeeded for app: " + appName);
24       return true;
25     } else {
26       LOGGER.error("maven build failed with exit code:" + exitCode);
27       return false;
28     }
```

```
    } catch (IOException | InterruptedException exception) {
      LOGGER.error("compileProject:maven build failed for app:" + appNam
      return false;
    }
  }
```

经过maven编译，编译的class文件默认会放在target/classes；如果是多模块代码，则代码会放在各自module下的target/classes，这时需要把各个模块的.class文件拷贝到一个目录下，方便后续JaCoCo做report操作。

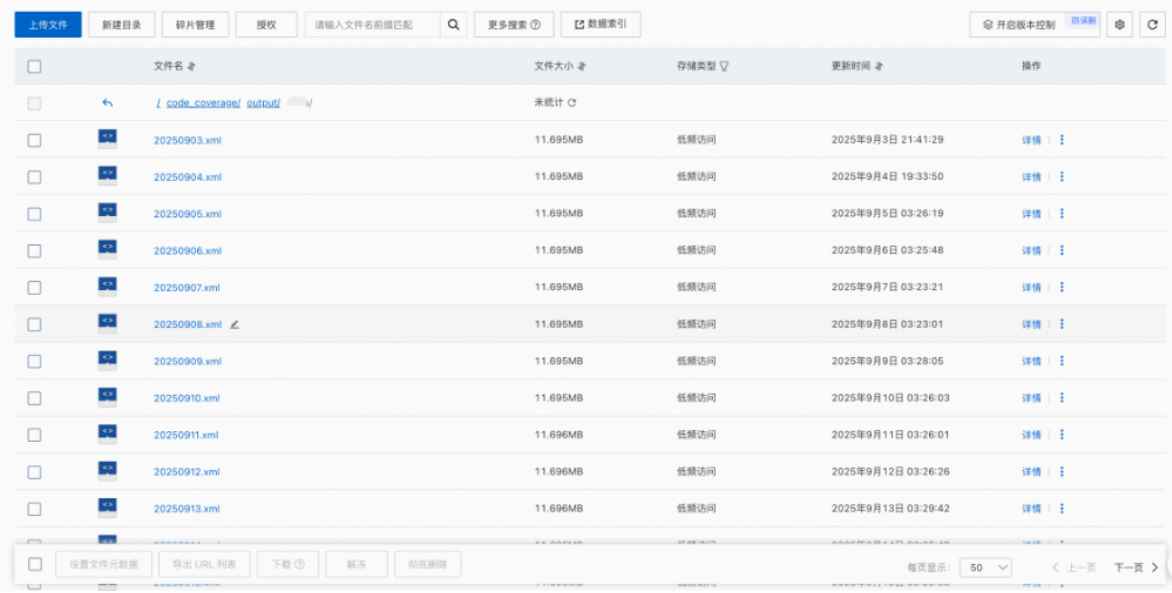
- 2.3.3 覆盖率报告生成

拿到了.exec采集数据和代码编译.class文件，最后一步就是将两者合并交给JaCoCo来生成详细的代码覆盖率文件的report操作。

```
/**
 * 创建XML格式的覆盖率报告
 *
 * @param execFileLoader 执行数据加载器
 * @param bundleCoverage 覆盖率分析结果
 * @param xmlPath        XML报告文件路径
 * @throws Exception 创建失败
*/
private void createXmlReport(ExecFileLoader execFileLoader, IBundleCove
  throws Exception {
  final List<IReportVisitor> visitors = new ArrayList<>();
  final XMLFormatter formatter = new XMLFormatter();
  visitors.add(formatter.createVisitor(Files.newOutputStream(Paths.get
  IReportVisitor reportVisitor = new MultiReportVisitor(visitors);
  reportVisitor.visitInfo(execFileLoader.getSessionInfoStore().getInfos
                          execFileLoader.getExecutionDataStore().getCor
  reportVisitor.visitBundle(bundleCoverage, null);
  reportVisitor.visitEnd();
}
```

这里IReportVisitor接收采集的数据和类文件，在visitBundle()时生成代码覆盖率数据，最终在指定文件目录下生成xml版本的报告。

xml的报告按应用维度生成，最后通过oss保存在各自应用目录下，使用日期作为报告名称。




覆盖率详细数据按天生成，可以用这些数据直接导入IDEA中查看代码执行情况，但这样还是太繁琐低效，接下来介绍如何将这些数据集成到IDEA插件中，直接在IDEA中查看采集的数据。

## 2.4 插件设计

IDEA插件的开发有一定门槛，涉及到很多intellij platform的概念，这里只做简单介绍，主要还是介绍插件功能的设计。涉及到的IDEA插件的概念是：Action、projectService、applicationConfigurable扩展点，详细可参考：IDEA插件开发。

**action:** 用于在IDEA内交互的行为处理，比如显示代码覆盖数据/隐藏代码覆盖率数据；

**projectService:** 主要的插件处理逻辑，可将产品功能封装到进来；
**applicationConfigurable:** 插件配置面板，支持插件配置的修改和持久化；
这些插件概念在代码覆盖率插件开发中都有涉及到，在实现部分会有具体介绍。

### 2.4.1 插件功能

插件要实现的功能包括：

- **打开&关闭代码执行覆盖展示**：手动打开或者关闭代码的覆盖率数据展示；
- **数据自动/手动下载**：在覆盖率数据展示时，自动下载oss的数据到本地(如果本地没有缓存数据)；
- **插件配置**：支持配置插件相关的参数，例如oss的配置、下载最近N天数据、缓存周期、下载超时等；
- **数据缓存**：支持将下载的数据缓存在本地，只要缓存未过期，优先使用本地数据。

### 2.4.2 插件实现

插件交互逻辑主要实现以Action为主，正如前面介绍，AnAction是用户行为处理的功能。插件主要实现了以下功能：




这些action、service和configure需在插件配置中注册生效。projectService和applicationConfigurable为IDEA的扩展点：

```
<!-- 插件扩展点 -->
<extensions defaultExtensionNs="com.intellij">
  <!-- 通知组 -->
  <notificationGroup id="JaCoCo Coverage Service" displayType="BALLOON"/>
  <notificationGroup id="JaCoCo Coverage Download" displayType="BALLOON"/>

  <!-- 项目组件 -->
  <projectService serviceImplementation="com.taobao.[redacted].jacoco.service.CoverageService"/>

  <!-- 配置界面 -->
  <applicationConfigurable instance="com.taobao.h[redacted].jacoco.settings.PluginConfigurable"/>

</extensions>
```

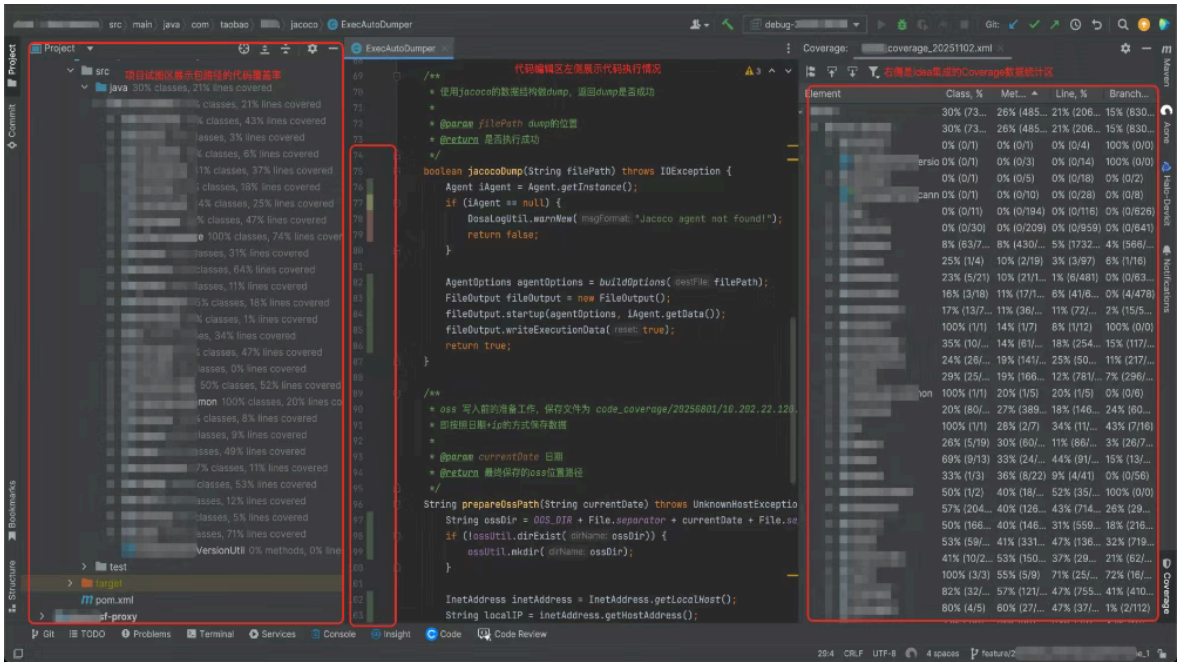
Action放在Actions标签下注册：

```xml
<!-- 动作组 -->
<actions>
  <!-- 主菜单组 - 二级菜单结构 -->
  <group id="iCoverage.MainMenu" text="代码覆盖率" description="工具栏代码覆盖率配置" popup="true">
    <add-to-group group-id="ToolsMenu" anchor="last"/>

    <!-- 交互行为操作子菜单 -->
    <group id="iMainMenuCoverage.SubMenu" text="覆盖率操作" description="工具栏覆盖率相关操作action">
      <action id="iMainMenuCoverage.ShowCoverage"
              class="com.taobao.h        .jacoco.actions.ShowCoverageAction"
              text="显示覆盖率"
              description="Show project coverage data"/>
      <action id="iMainMenuCoverage.HideCoverage"
              class="com.taobao.h        .jacoco.actions.HideCoverageAction"
              text="隐藏覆盖率"
              description="Hide project coverage data"/>
      <separator/>
      <action id="iMainMenuCoverage.RefreshCoverageData"
              class="com.taobao.h        .jacoco.actions.RefreshCoverageAction"
              text="刷新覆盖率"
              description="Refresh coverage from oss file"/>
      <action id="iMainMenuCoverage.OpenSettings"
              class="com.taobao.h        .jacoco.actions.OpenSettingsAction"
              text="覆盖率配置"
              description="Open panel of coverage settings"/>
      <action id="iMainMenuCoverage.MultiDayCoverageInfo"
              class="com.taobao.h        .jacoco.actions.OpenCoverageInfoAction"
              text="覆盖率数据信息"
              description="Refresh coverage from oss file"/>
    </group>
  </group>
```

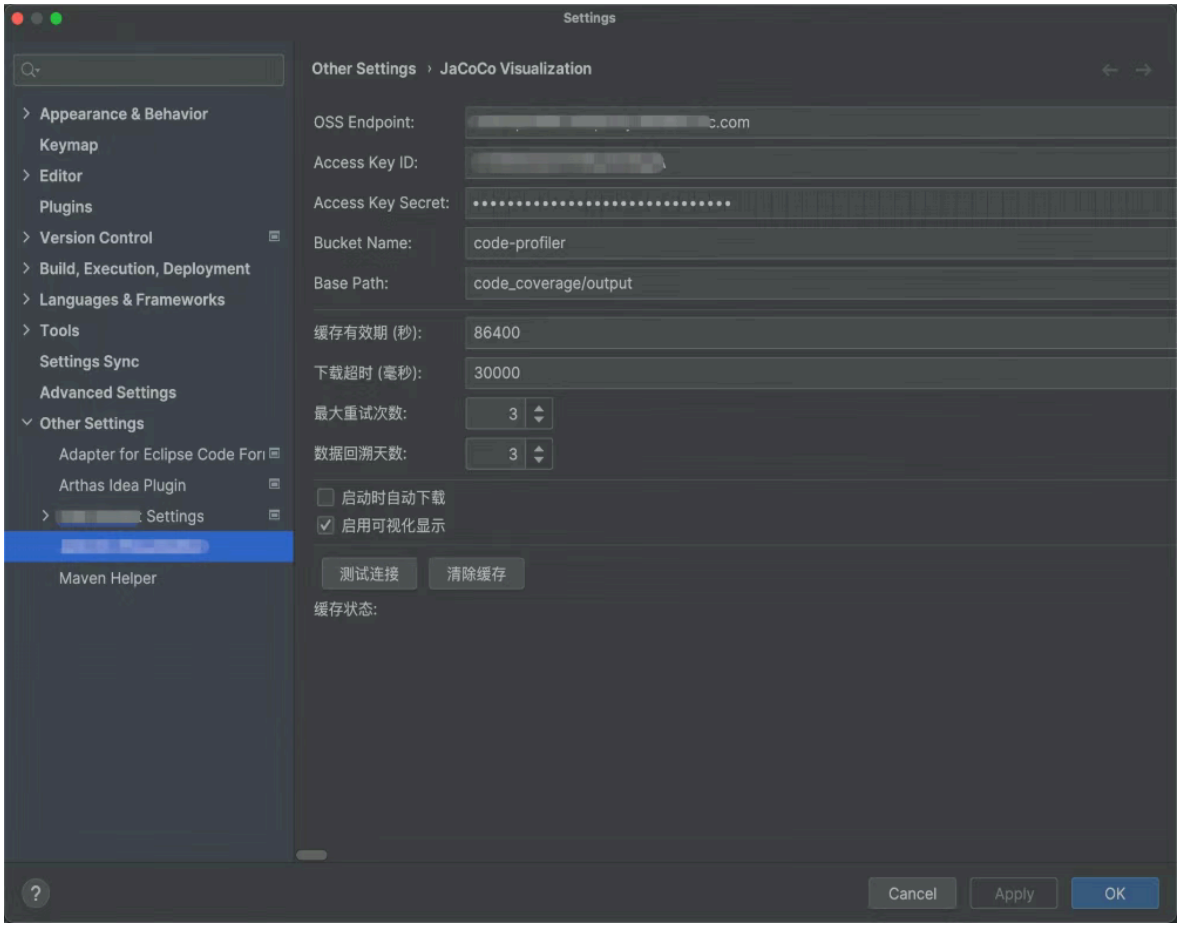
- 2.4.3 插件效果

在打开的代码编辑器右键(macOS双指单击)，展示注册的action按钮，具备【显示刷新率】、【隐藏刷新率】和【刷新率覆盖】三个功能按钮。



在项目视图展示的是按package统计的代码覆盖率数据，可以快速导航到覆盖率低的路径做代码清理；中间编辑器的左边框展示了代码行的执行情况，针对每行可执行的部分，绿色标识代码有执行，红色标识未执行过，黄色标识部分覆盖，这里可用来判断线上代码实际执行的分支；右侧是Coverage面板，详细展示了类、方法、行和分支覆盖率数据，可快速筛选指定包路径。

在顶部Tools下可以找到更多代码覆盖率的action，例如刷新数据、打开插件配置和展示覆盖率数据信息等。以下为配置面板：

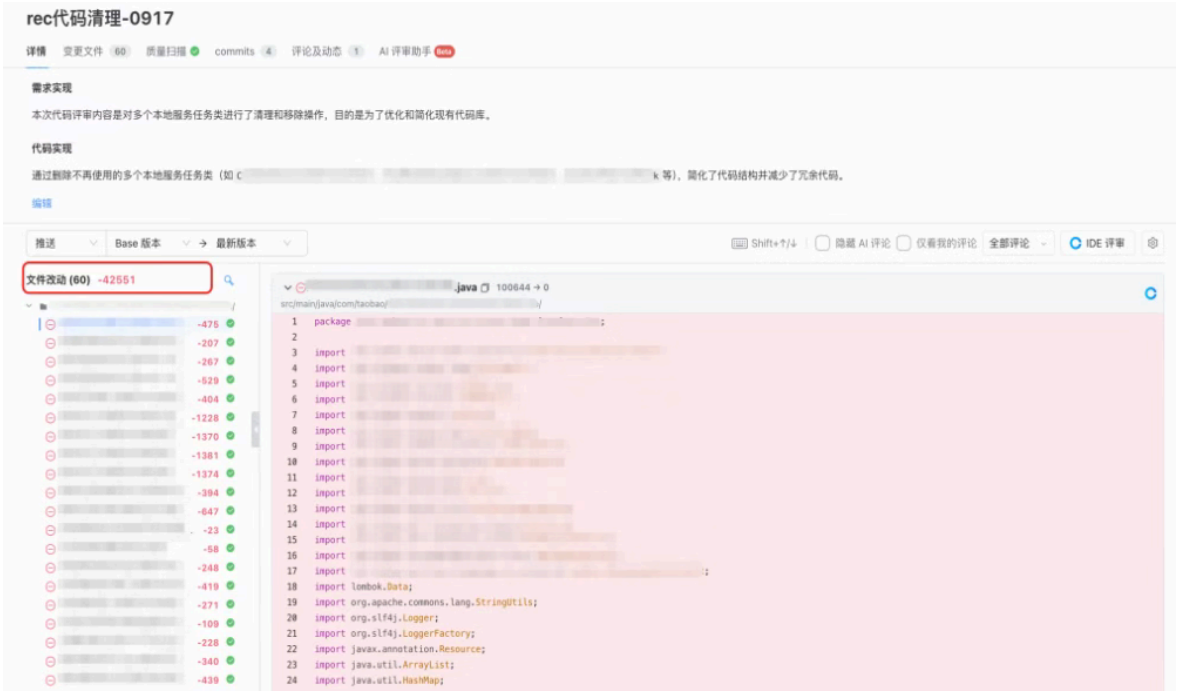



在面板上可以配置下载数据的oss配置，缓存有效期、用于分析的文件数等。以上即为插件的核心功能，通过插件极大加速了无效代码的清理，使得代码清理准而快。

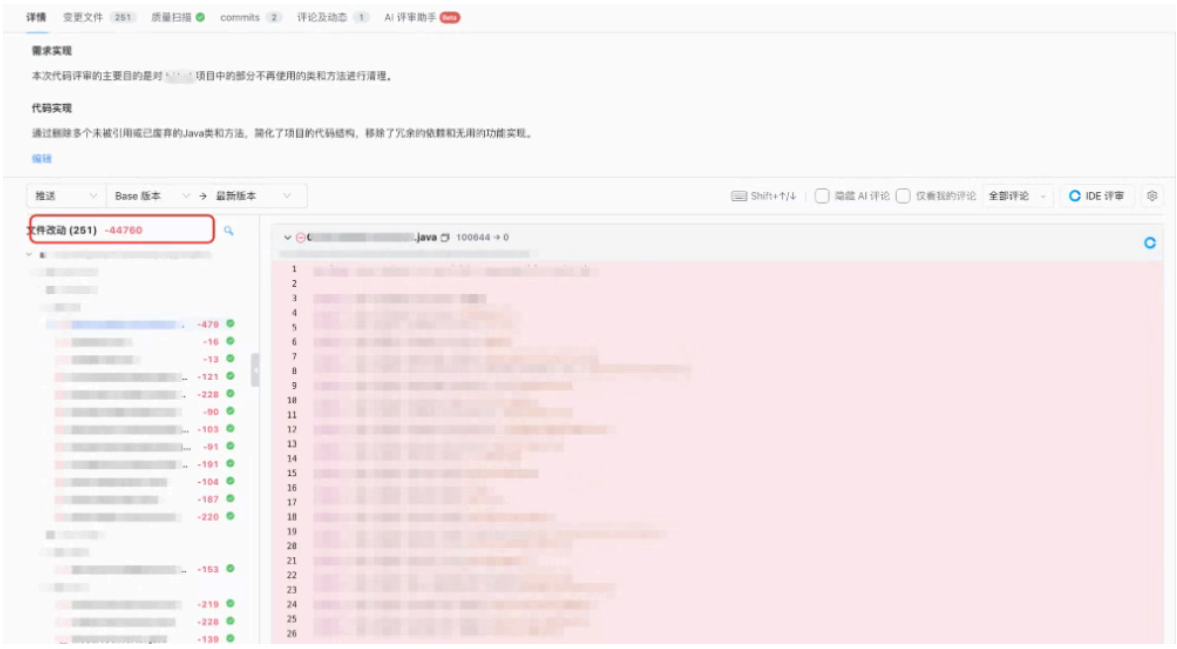
## 03 治理效果

基于代码执行的数据和可视化插件，代码清理工作非常高效，列举批量代码清理工作的过程。

**R代码清理**：




**B代码清理：**




通过类似上述的代码清理过程，对业务域内的主要的应用做无效代码清理的效果：

| 应用 | 代码基线 | 清理代码 | 降低比例 |
| --- | --- | --- | --- |
| B | 21.5w | 15.4w | 71% |
| R | 43.1w | 18.7w | 43% |
| D | 25.2w | 2.1w | 8.3% |

其中B应用清理的效果最佳，因其与R应用存在较多代码冗余；R应用其次，目前主要的业务高频迭代在R应用，代码量也是最大的，还有一定清理空间；而D应用代码因被B应用和R应用所依赖，所以最后清理，这部分工作还在进行中。




通过代码插桩染色和执行情况的采集，较好对D应用代码做了治理，中间过程有收获也走了弯路。

过程收获：

- 探索了JaCoCo工作原理，学习其优秀的代码设计(主要基于访问者模式，结合asm类代码修改)，最终借鉴其框架实现了代码执行覆盖的采集；
- D应用代码历史悠久，通过对执行数据的分析和代码业务的判断，实现了对D应用无效代码的规模化清理，整体过程对业务无感；
- 学习了IDEA插件开发框架，完成自己第一个IDEA插件的开发，过程中有各种探索，为了探索IDEA的工作原理，debug过IDEA源码，感兴趣的可以去IDEA社区版git仓库查看（下文附链接）。

走的弯路：

- 前期没有深入了解热部署类加载原理和版本管理，导致对部署代码的覆盖采集问题出现偏差，误导了问题的定位方向；
- 对使用AI agent完整开发IDEA组件前期期望过高，利用AI从设计到代码开发做完整插件实现，存在一些问题：
  - 前期设计的方案后续每做一次需求调整，AI的修改都有可能让代码设计更加复杂，这种情况因token限制导致下次对话上下文丢失更加严重；
  - IDEA平台内部的代码实现代码对AI偏黑盒，经常在代码库检索不到，想让AI复用IDEA内部接口功能较难实现；
  - 对IDEA插件领域模型不熟悉的话，agent生成的代码逻辑自己都不一定搞懂原理，更别说后续做问题定位和插件升级。
  - 而最终版插件的实现方案，还是人工通过调试IDEA社区版源码让IDEA的Coverage插件逻辑白盒化，使用原生代码覆盖率的接口实现了插件核心逻辑，而像bug的定位&修复等工作使用AI确实反而高效，算是“锦上添花”了。
- 采集数据基于安全生产环境，结合业务功能清理代码，基本不会影响线上核心链路，自是采集过程没有完全覆盖线上请求，诸如冷链路、大促链路、老版本逻辑让人防不胜防，清理过程也有遇到清理的类仍有少量流量的情况。

刚来团队时就发现D应用代码有很多历史悠久的代码，当时就有做无效代码清理的想法，直到我逐渐加深对D代码和技术框架的原理，才真正着手做这件事。前期有调研相关产品，最终基于D的情况做了技术方案设计和代码的开发，确实下线清理了不少代码，也将在后续继续做治理优化。虽然当前方案只支持了D应用代码治理，也并非所有应用都需要做代码治理，但这套方案可以迁移到需要的其它业务，尤其是历史代码包袱重且难以重构的，希望能给需要的同学提供一种可行方案。





- JaCoCo agent：https://www.eclemma.org/jacoco/trunk/doc/agent.html
- JaCoCo cli：https://www.eclemma.org/jacoco/trunk/doc/cli.html
- Action：https://plugins.jetbrains.com/docs/intellij/plugin-actions.html?from=DevkitPluginXmlInspection
- projectService：https://plugins.jetbrains.com/docs/intellij/plugin-services.html?from=DevkitPluginXmlInspection
- applicationConfigurable：https://plugins.jetbrains.com/docs/intellij/plugin-extensions.html?from=DevkitPluginXmlInspection#exploring-available-extensions
- IDEA插件开发：https://plugins.jetbrains.com/docs/intellij/plugins-quick-start.html?from=DevkitPluginXmlInspection
- git仓库：https://github.com/JetBrains/intellij-community/tree/pycharm/233.15619.17?tab=readme-ov-file#readme





本文作者星言，来自淘天集团-首页&信息流工程团队。团队负责淘宝首页和信息流的工程技术。我们的目标是为数亿用户构建一个体验极致、高并发、高可用、高效能的在线系统；通过工程架构和数据链路的升级，优化系统性能、提升迭代效率、突破算法模型天花板；通过大模型技术在用户理解、推荐算法、产品创新的深度应用，提升用户体验和流量效率。

巨大的用户流量、深度的商业思维、前沿的算法工程、核心的系统架构，种种挑战和机会也欢迎你的加入！

## 拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法




**大淘宝技术**

大淘宝技术官方账号

967篇原创内容

公众号


阅读 1.7万

**留言**

写留言

还没有留言，来写首评